A
Project Report
on
SmartDesk
AI Automated Ticket Classification & Routing

Submitted By

RAMISETTY MANJU BHARGAV

S SANIYA BEGUM

Under the Guidance of

Dr. PENUGONDA RAVI KUMAR

M.E. (IISc Bangalore), Ph.D. (University of Aizu, Japan)

Assistant Professor

Computer Science and Engineering

RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES (APIIIT)

R K VALLEY, VEMPALLI, YSR (Dist.) – 516330

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

2025-2026

Rajiv Gandhi University of Knowledge Technologies

RK Valley, YSR (Dist.), Andhra Pradesh - 516330

CERTIFICATE

This is to certify that the project report titled **"SmartDesk AI Automated Ticket Classification & Routing"**, carried out by **Ramisetty Manju Bhargav(R200158),S Saniya Begum(R201085)** has been completed under my guidance and supervision. This report is submitted to the **Department of Computer Science and Engineering** in partial fulfilment of the requirements for the **Major Project - 2**, as part of the curriculum for the **Bachelor of Technology in Computer Science and Engineering** during the **Academic Year 2025–2026**. The work embodied in this report has been reviewed and is found to be in accordance with the academic requirements of the University.

Signature of Internal Guide
Dr. P. RAVI KUMAR
M.E. (IISc Bangalore)
Ph.D. (University of Aizu, Japan)
Assistant Professor
Computer Science and Engineering
RGUKT RK VALLEY

Signature of HOD
Dr. CH. RATNA KUMARI,
M.Tech. (University of Hyderabad)
Ph.D. (JNTU Hyderabad)
Head of the Department
Computer Science and Engineering
RGUKT RK VALLEY

Signature of External Examiner

DECLARATION

I hereby declare that the project work titled **“SmartDesk AI Automated Ticket Classification & Routing”**, submitted to the **Department of Computer Science and Engineering**, is a genuine piece of work carried out by our team under the guidance of **Dr. P. Ravi Kumar**, *M.E.*, *Ph.D*. This report is submitted in partial fulfilment of the requirements for the **Major Project - 2**, as part of the curriculum for the **Bachelor of Technology in Computer Science and Engineering** during the **Academic Year 2025–2026**.

We further declare that this work has not been submitted elsewhere for the award of any other degree or diploma.

Date:

Place: RK Valley

Project Team

Ramisetty Manju Bhargav(R200158)

S Saniya Begum(R201085)

ACKNOWLEDGEMENT

I would like to express our sincere gratitude to all those who supported and guided us throughout the successful completion of this Major Project - 2 titled **“SmartDesk AI Automated Ticket Classification & Routing”**

We are deeply thankful to our project guide, **Dr. P. Ravi Kumar,** *M.E., Ph.D.*, for his invaluable guidance, constant encouragement, and constructive suggestions at every stage of the project. His mentorship played a crucial role in shaping the direction and outcome of our work.

We extend our heartfelt thanks to **Dr. Ch. Ratna Kumari,** *M.Tech., Ph.D.*, **Head of the Department, Computer Science and Engineering**, for her continuous support and for providing us with the resources and academic environment necessary for this project.

We sincerely acknowledge and express our thankfulness for the academic support of **Mr. D. Ramesh Kailas,** *M.Tech.*, **Dean of Academics, RGUKT RK Valley**.

We are also grateful to **Prof. A. V. S. S. Kumara Swami Gupta,** *M.Tech., Ph.D.*, **Director of RGUKT RK Valley**, for his encouragement and for fostering a culture of research and innovation.

Our sincere thanks go to the **faculty members of the Department of Computer Science and Engineering** for their guidance, and academic support.
Finally, we appreciate the collaborative efforts of our team members and the support extended by our peers, which made this project a rewarding learning experience.

Ramisetty Manju Bhargav(R200158)

S Saniya Begum(R201085)

TABLE OF CONTENTS

CH.NO.	INDEX	PAGE NO.
	CERTIFICATE	ii
	DECLARATION	iii
	ACKNOWLEDGEMENT	iv
	ABSTRACT	1
	LIST OF FIGURES	2
1	**INTRODUCTION** 1.1 Background 1.2 Need for Automated Ticket Classification 1.3 Challenges in Manual and Rule-Based Classification 1.4 Emergence of AI-Driven Text Understanding in ITSM 1.5 Motivation for the Present Work 1.6 Problem Statement 1.7 Objectives of the Project 1.8 Major Contributions of the Project 1.9 Conclusion	3–7
2	**LITERATURE REVIEW** 2.1 Machine Learning Approaches for Text Classification 2.2 Deep Learning Models for Semantic Text Understanding 2.3 Transformer Framework and its Significance in NLP 2.4 BERT: Bidirectional Transformer for Text Classification 2.5 Workflow Automation Systems in Operational Environments 2.6 Summary of Literature Review	8–11
3	**SYSTEM DESIGN AND ARCHITECTURE** 3.1 Overview of the Proposed System 3.2 Dataset Description 3.3 Class Distribution 3.4 Preprocessing and Text Cleaning 3.5 BERT Model Architecture 3.6 System Modules and Their Roles 3.7 Summary	12-17

4	**METHODOLOGY** 4.1 Overview of the Methodology 4.2 Baseline Model Training 4.3 BERT Model Fine-Tuning 4.4 Evaluation Metrics 4.5 Confusion Matrix 4.6 Deployment Using Flask 4.7 Workflow Automation Using n8n 4.8 Summary	18-23
5	**IMPLEMENTATION** 5.1 Model Implementation 5.2 API Development Using Flask 5.3 Workflow Automation Using n8n 5.4 System Integration and Testing 5.5 User Interface and Output Screens 5.6 Summary	24-28
6	**RESULTS AND EVALUATION** 6.1 Evaluation Metrics 6.2 Baseline Model Results 6.3 BERT Model Performance 6.4 Confusion Matrix Analysis 6.5 Comparative Analysis 6.6 Summary	29-32
7	**CONCLUSION AND FUTURE WORK** 7.1 Conclusion 7.2 Future Work 7.3 Summary	33-34
	REFERENCES	35

ABSTRACT

The rapid growth of digital services has significantly increased the volume of IT support requests generated in modern organizations. Traditional manual classification of support tickets is slow, inconsistent, and inefficient, resulting in delayed issue resolution and degraded service quality. With the growing emphasis on delivering superior customer experience and meeting strict Service Level Agreements (SLAs), there arises a crucial need for intelligent, automated systems that can accurately classify and route support tickets in real time.

This project presents an end-to-end automated support ticket classification and routing pipeline leveraging state-of-the-art transformer-based Natural Language Processing (NLP) and low-code workflow automation. The proposed system integrates three key components designed to improve semantic understanding, operational efficiency, and real-world deployability:

1. A BERT-based text classification model fine-tuned on a large customer support ticket dataset, enabling deep contextual analysis and improved accuracy over traditional machine learning approaches;
2. A Flask-based REST API deployment enabling seamless integration with external systems;
3. An n8n workflow automation pipeline that processes model predictions, generates structured outputs, and automatically routes categorized tickets to respective IT support teams via Telegram.

This work highlights the synergy between modern NLP architectures and workflow automation tools, showcasing a scalable, practical, and deployable solution for intelligent IT Service Management (ITSM). The project contributes toward building smarter organizational support infrastructures that align with evolving business needs and customer expectations.

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 3.1	Overall System Workflow Architecture	13
Figure 3.2	Sample of the Cleaned Customer Support Ticket Dataset	14
Figure 3.3	Class Distribution Before Oversampling	14
Figure 3.5	BERT Model Architecture	16
Figure 4.1	Confusion Matrix of the Fine-Tuned BERT Model	21
Figure 4.7	n8n Workflow for Automated Ticket Routing	22
Figure 5.3	n8n Workflow	25
Figure 5.5.1	Direct Ticket Classification Interface	27
Figure 5.5.2	n8n Form Submission for Batch Ticket Processing	28
Figure 6.4	Confusion Matrix of BERT Model	31

CHAPTER 1

INTRODUCTION

1.1 Background

Information Technology Service Management (ITSM) has become a critical component in modern organizations, supporting day-to-day operations, digital workflows, and service continuity. As businesses increasingly rely on digital infrastructure, the number of user-generated support tickets has grown exponentially. These tickets may include software faults, service interruptions, access requests, system behavior anomalies, configuration requirements, and various service queries. Each ticket must be categorized accurately so that it reaches the right support team responsible for addressing that specific type of issue. Traditionally, this categorization is performed manually by support agents who carefully read each ticket and assign it to a predefined class such as *Incident*, *Problem*, *Change*, or *Request*.

However, as IT infrastructure scales and the complexity of service environments increases, manual classification becomes inefficient. Human triaging is slow, error-prone, and inconsistent, leading to delays, misrouting, and significantly increased operational workloads. In large organizations, even a small misclassification rate can result in substantial productivity loss and violation of SLA (Service Level Agreement) commitments. This situation highlights the urgent need for intelligent, scalable, and automated solutions that can efficiently handle support ticket classification.

1.2 Need for Automated Ticket Classification

Support teams handle issues ranging from minor user requests to high-priority incidents. Manual triage suffers from the following limitations:

1. Scalability issues

As ticket volume grows, human triage becomes a bottleneck.

2. Human inconsistency

Two agents may classify the same ticket differently.

3. Misrouting delays

A wrongly classified ticket may be passed between teams multiple times.

4. High operational cost

Organizations require more manpower for classification.

5. SLA breaches

Classification delays cascade into longer resolution times.

Automatic classification using AI offers:

- Faster triaging
- Uniform classification
- Reduced manual workload
- Better adherence to SLAs
- Improved customer satisfaction

1.3 Challenges in Manual and Rule-Based Classification

Despite being the oldest form of ticket triage, manual classification poses substantial limitations. Human agents often deal with large backlogs, leading to delayed decisions and increased fatigue, which further increases error rates. Additionally, human interpretation is inherently subjective—two agents might classify the same ticket differently depending on their experience or understanding of the issue. Misclassification results in tickets being assigned to the wrong teams, triggering rerouting, further delays, and a cycle of inefficiency that negatively affects users awaiting support.

Early attempts to improve this process utilized rule-based systems, where keyword matching and manually crafted heuristics attempted to automate classification. While suitable for simple cases, rule-based approaches break down quickly in real-world environments. IT support tickets often contain ambiguous or multi-intended language, making it difficult to represent them using fixed rules. They may include domain-specific terminology, nested explanations, and varying descriptions even for similar problems. Rule-based systems also fail to interpret negation, sarcasm, or contextual details. Maintaining such rule sets becomes cumbersome as new types of issues and technologies emerge. These limitations highlight the need for a more robust, context-aware, and scalable approach.

1.4 Emergence of AI-Driven Text Understanding in ITSM

The evolution of Natural Language Processing (NLP) has opened new possibilities for automating ticket classification. Traditional machine learning methods, such as Logistic Regression and Support Vector Machines, represented early efforts to learn patterns from textual data. These models transformed text into numerical vectors using methods such as

TF-IDF and bag-of-words. While useful, they inherently lack semantic understanding because they treat each word independently, ignoring context and relationships within the sentence.

Deep learning marked a significant improvement over traditional methods. Models like LSTMs and CNNs enhanced the ability to capture sequential and local patterns. However, these models still struggled to understand long-range dependencies and the deeper contextual meaning essential for interpreting complex IT support tickets.

The introduction of Transformer-based architectures revolutionized NLP by enabling models to analyze text bidirectionally and understand relationships between words regardless of their position in the sentence. BERT (Bidirectional Encoder Representations from Transformers), in particular, has demonstrated remarkable performance in multiple text classification tasks due to its superior semantic understanding. It captures context with high fidelity, making it exceptionally suitable for classifying ITSM tickets that often contain technical language, multi-step problem descriptions, and implicit meanings. This advancement forms the foundation for the automated system proposed in this project.

1.5 Motivation for the Present Work

The motivation behind this project arises from the clear gap between existing ticket classification approaches and the demands of modern IT support environments. While previous research—particularly the base paper using Google Vertex AutoML—demonstrated the potential of automated ticket categorization, several limitations remained unaddressed. AutoML models, despite achieving reasonable accuracy, lack deep contextual understanding of technical phrases and multi-layered descriptions. Moreover, the base paper focused solely on model training and evaluation without offering any insights into real-world deployment or integration into operational workflows.

In practical ITSM scenarios, classification alone is insufficient; organizations require complete automation pipelines that not only classify but also route tickets to appropriate teams instantly. Additionally, AutoML systems do not easily support explainability, customization, or advanced semantic modeling that transformers can provide. Recognizing these limitations, this project aims to develop a more capable system using BERT, a powerful transformer-based NLP model, and integrate it with workflow automation using n8n. This integration allows for automated routing, real-time notifications, and immediate operational

action, thereby bridging the gap between machine learning research and practical IT support solutions.

1.6 Problem Statement

Modern IT support systems generate large volumes of unstructured textual data that must be accurately categorized for efficient processing. Existing methods for ticket classification—both manual and automated—lack the scalability, contextual understanding, and integration capabilities required for real-time ITSM operations. Misclassification leads to operational delays, reduced productivity, and potential SLA violations.

Therefore, the problem addressed by this project is: **"To design and implement an automated IT support ticket classification and routing system that uses transformer-based NLP (BERT) and workflow automation (n8n) to achieve real-time, scalable, and accurate ticket triage."**

1.7 Objectives of the Project

The primary goal of this project is to build an intelligent system capable of classifying and routing support tickets automatically. To achieve this, several specific objectives are formulated:

- To preprocess and analyze customer support ticket datasets, preparing them for machine learning and transformer models.
- To implement baseline machine learning models (Logistic Regression and SVM) and compare their performance against BERT.
- To fine-tune a transformer-based BERT model that can understand complex IT-related language and achieve high classification accuracy.
- To deploy the trained model using a Flask API, enabling external systems to request predictions seamlessly.
- To design an n8n workflow pipeline that automates routing of classified tickets to the correct support teams via Telegram.
- To evaluate the complete system using industry-standard metrics such as precision, recall, F1-score, and accuracy.
- To build a fully functional solution suitable for real-world ITSM automation.

1.8 Major Contributions of the Project

This project makes several important contributions to the domain of ITSM automation. First, it presents a transformer-based ticket classification model capable of understanding the deeper semantics of technical support descriptions, offering significantly improved accuracy over traditional approaches. Second, it introduces a full deployment pipeline that exposes the BERT model as a REST API, making it accessible to any external workflow or support tool. Third, the integration of n8n automation provides a novel mechanism for real-time routing, file organization, and ticket dissemination to support teams across platforms such as Telegram. Finally, the project delivers a comprehensive comparative analysis between classical ML models, AutoML approaches from existing literature, and modern transformer-based architectures, demonstrating the superior performance and practical viability of the proposed system.

1.9 Conclusion

The expansion of digital infrastructures and services has increased the demand for sophisticated, reliable, and automated methods for handling IT support tickets. Traditional manual and rule-based approaches are no longer adequate in environments requiring real-time classification, high accuracy, and scalability. Machine learning and deep learning models offered partial solutions but still lacked the contextual understanding required for real ticket descriptions. This chapter has identified the pressing need for an automated, context-aware ticket classification system and outlined the motivation for adopting modern NLP models such as BERT. It also highlighted gaps in existing research, particularly the limitations of AutoML-based approaches that do not incorporate semantic modeling or workflow automation.

The introduction establishes the foundation for a comprehensive investigation into the integration of transformer-based models with automated routing pipelines. The remaining chapters build upon this groundwork by reviewing related literature, describing the methodology and system architecture, discussing implementation details, and presenting experimental results that demonstrate the effectiveness of the proposed solution.

CHAPTER 2
LITERATURE REVIEW

The field of automated text classification has seen substantial growth over the last decade, driven primarily by advancements in machine learning, deep learning, and more recently, transformer-based architectures. As organizations increasingly depend on digital communication channels for support and service operations, the volume and complexity of textual data have intensified. This exponential rise has transformed text classification into a fundamental requirement for efficient information processing. This chapter presents a comprehensive review of literature related to text representation techniques, machine learning models, deep learning networks, transformer architectures, and workflow automation technologies. The objective is to establish the foundation upon which the proposed automated support ticket classification and routing system is developed.

2.1 Machine Learning Approaches for Text Classification

Early attempts at automating text classification primarily relied on traditional machine learning techniques. These models treated text as a numerical representation using handcrafted features generated through methods such as Bag-of-Words (BoW), Term Frequency–Inverse Document Frequency (TF-IDF), and n-gram statistics. Despite their simplicity, these feature extraction techniques played a crucial role in enabling classical models to interpret textual data.

Among the most commonly used algorithms, **Logistic Regression** emerged as a strong baseline due to its computational efficiency and ability to handle high-dimensional vector spaces. Its linear decision boundary, while effective for simple classification tasks, often fails to capture complex relationships in longer, context-rich sentences.

Support Vector Machines (SVMs) demonstrated better robustness by maximizing the margin between classes and performing well on sparse TF-IDF vectors. They were widely adopted for spam filtering, sentiment analysis, and ticket classification tasks in early NLP systems. However, SVMs remain limited by their inability to understand semantics or word order, relying solely on statistical patterns within feature vectors.

While these models offered a foundation for text classification, their limitations became evident as applications demanded deeper contextual understanding and improved generalization across diverse writing styles and vocabularies. These shortcomings motivated the transition to deep learning-based architectures.

2.2 Deep Learning Models for Semantic Text Understanding

The introduction of deep learning marked a significant evolution in NLP research. Instead of relying on manually engineered features, deep learning models learned hierarchical representations directly from raw text, enabling more nuanced and context-aware classification.

One of the earliest breakthroughs came from **Recurrent Neural Networks (RNNs)**, which processed input sequences step by step, maintaining a hidden state that captured transitions between words. Although RNNs improved the understanding of sequential dependencies, they suffered from vanishing gradient issues, limiting their ability to learn long-range relationships.

To address this limitation, **Long Short-Term Memory (LSTM)** networks were introduced. LSTMs employed gating mechanisms that allowed them to retain important information over longer sequences. They became widely used in tasks such as sentiment analysis, document classification, and intent detection. However, even LSTMs faced challenges when dealing with very long text descriptions or multiple paragraphs, as found in many real-world support ticket systems.

In parallel, **Convolutional Neural Networks (CNNs)** were adapted for text classification by treating sequences of word embeddings as spatial features. CNNs excelled at capturing local patterns and n-gram-level relationships but struggled with global context across long sentences.

Although these deep learning models offered significant improvements over traditional machine learning approaches, they lacked the ability to model long-range dependencies bidirectionally. Their reliance on sequential processing also made training computationally expensive and less scalable for large datasets. These limitations paved the way for the next major shift in NLP — transformer-based models.

2.3 Transformer Framework and its Significance in NLP

The transformer architecture revolutionized NLP by removing the need for sequential processing entirely. Instead of RNN-style recurrence, transformers utilize **self-attention mechanisms**, enabling the model to focus on all words in a sentence simultaneously. This design allows transformers to capture long-range dependencies, contextual meaning, and hierarchical relationships far more effectively than previous architectures.

Transformers quickly became the foundation of modern NLP research due to their ability to perform parallel computation and model complex interactions between words. They outperform earlier models in various NLP tasks including classification, machine translation, summarization, and question answering.

2.4 BERT: Bidirectional Transformer for Text Classification

Among transformer-based models, **BERT (Bidirectional Encoder Representations from Transformers)** stands out as one of the most influential advancements. Introduced by Google in 2018, BERT leverages bidirectional attention, meaning it analyzes text from both left and right contexts simultaneously. This capability enables a deeper understanding of nuanced meaning that was previously unattainable by machine learning or traditional deep learning models.

BERT is pre-trained on large corpora using two tasks:

1. **Masked Language Modeling (MLM)** – predicting missing tokens
2. **Next Sentence Prediction (NSP)** – understanding sentence coherence

These tasks provide BERT with a rich understanding of language structure and semantics. Fine-tuning BERT for specific downstream tasks such as ticket classification requires only minimal training, making it extremely powerful for domain adaptation.

In support ticket classification, BERT offers substantial advantages:

- It interprets technical language more accurately.
- It understands negation, multi-step reasoning, and conditional logic.
- It handles long and detailed descriptions with high precision.
- It achieves significantly higher accuracy compared to traditional ML models.

Because of these characteristics, BERT has become a preferred architecture for enterprise text processing systems.

2.5 Workflow Automation Systems in Operational Environments

While NLP models provide strong prediction capabilities, modern operational systems require end-to-end pipelines that integrate classification with automated actions. Workflow automation platforms such as **n8n**, **Zapier**, and **Airflow** allow organizations to automate tasks like routing, notification, escalation, and reporting without extensive programming.

In IT service environments, workflow automation plays a critical role by enabling:

- Automatic forwarding of classified tickets to the appropriate teams
- Real-time notifications and updates
- File processing and report generation
- Integration with communication tools such as email, Slack, or messaging APIs
- Reduction of manual triage workload

Despite the growing adoption of workflow systems in many industries, research literature indicates that combining workflow automation directly with NLP-based ticket classification remains relatively unexplored. This underlines the importance of developing integrated solutions — something this project achieves by connecting a BERT classifier with an n8n automation pipeline.

2.6 Summary of Literature Review

The literature reviewed in this chapter highlights the progression from traditional machine learning approaches to advanced deep learning and transformer models used for text classification. Classical ML techniques, while foundational, lack the capacity for true semantic understanding. Deep learning models provided improvements but still fell short in capturing long-range contextual dependencies. Transformer-based models, especially BERT, have set new standards in NLP due to their deep contextual modeling capabilities.

Additionally, workflow automation tools have emerged as essential components in modern IT operations, yet their integration with NLP-based classification systems remains limited in research. By understanding these developments, this chapter establishes the technological context necessary for designing the automated support ticket classification and routing system proposed in this work.

CHAPTER 3

SYSTEM DESIGN AND ARCHITECTURE

The design of the proposed automated ticket classification system focuses on building a reliable, scalable, and efficient pipeline that can process raw support tickets, classify them into appropriate categories, and route them to specific support teams automatically. This chapter explains the architectural design of the system, the dataset used, preprocessing operations, the role of the BERT model, and how the components interact to form a complete ticket automation workflow.

3.1 Overview of the Proposed System

The entire system is structured as a sequence of interconnected components that work together to perform automatic classification and routing. It begins with raw ticket data, which is cleaned and preprocessed before being fed into machine learning models. While baseline models like Logistic Regression and SVM are used for comparison, the core of the system relies on the BERT transformer model for final prediction. Once the BERT model predicts the category, the results are sent to a Flask API, which is connected to an n8n workflow. The workflow then delivers the classified tickets to appropriate Telegram groups.

In simple terms, the system follows this flow:

1. **Input Ticket Data**
2. **Preprocessing and Cleaning**
3. **Classification using BERT**
4. **Prediction via API**
5. **Automatic Routing through n8n**

This ensures that support tickets are classified instantly and delivered to the correct team without manual effort.

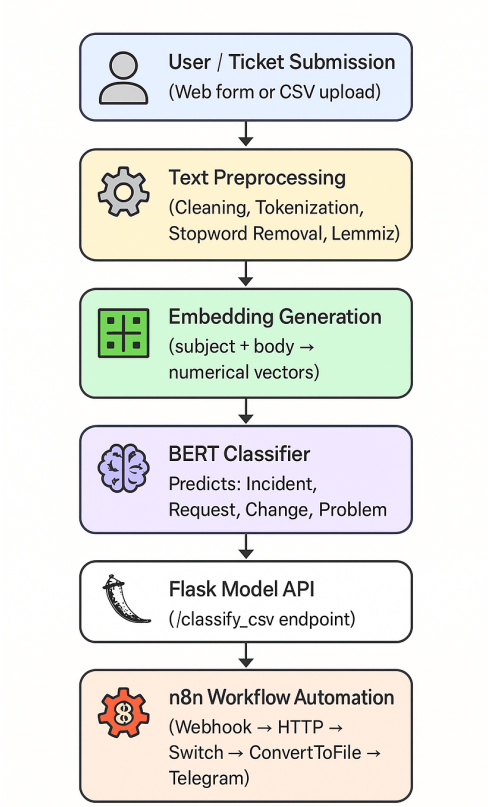

Figure 3.1 –Overall System Workflow Architecture

3.2 Dataset Description

The dataset used for this project consists of approximately **28,000 real-world customer support tickets** covering a wide range of technical and service-related issues. Each ticket originally contained several fields such as *subject*, *description*, *priority*, and *queue*. For classification purposes, the **subject** and **description** fields were merged and cleaned to form a consolidated feature called **cleaned_text**, which represents the final input to the machine learning models.

The dataset includes four primary ticket types: **Incident, Problem, Change, and Request**. These categories represent common IT Service Management (ITSM) workflows and are widely used in industry-grade ticketing platforms.The tickets vary significantly in length, vocabulary, and grammatical structure, making them ideal for evaluating the semantic capabilities of transformer-based models like BERT.

A sample of the cleaned dataset used in this study is shown below:

	cleaned_text	type	priority	queue
0	account disruption dear customer support teamn...	Incident	high	Technical Support
1	query smart home system integration features d...	Request	medium	Returns and Exchanges
2	inquiry regarding invoice details dear custome...	Request	low	Billing and Payments
3	question marketing agency software compatibili...	Problem	medium	Sales and Pre-Sales
4	feature query dear customer supportnni hope me...	Request	high	Technical Support
...	...	...	...	...
28256	guidelines securing medical data obs studio se...	Request	high	Technical Support
28257	provide information digital strategies boost b...	Request	medium	Billing and Payments
28258	support marketing enhancements request assista...	Change	high	Technical Support
28259	assistance needed ifttt docker integration fac...	Problem	low	Technical Support
28260	hello customer support inquiring billing optio...	Request	low	Billing and Payments

Figure 3.2 – Sample of the Cleaned Customer Support Ticket Dataset

3.3 Class Distribution

Real-world datasets often suffer from class imbalance, meaning some classes appear far more frequently than others. In this dataset, categories like **Incident** and **Request** are more common, whereas **Change** and **Problem** appear less frequently. To prevent the model from becoming biased, oversampling is applied to minority classes to balance the dataset.

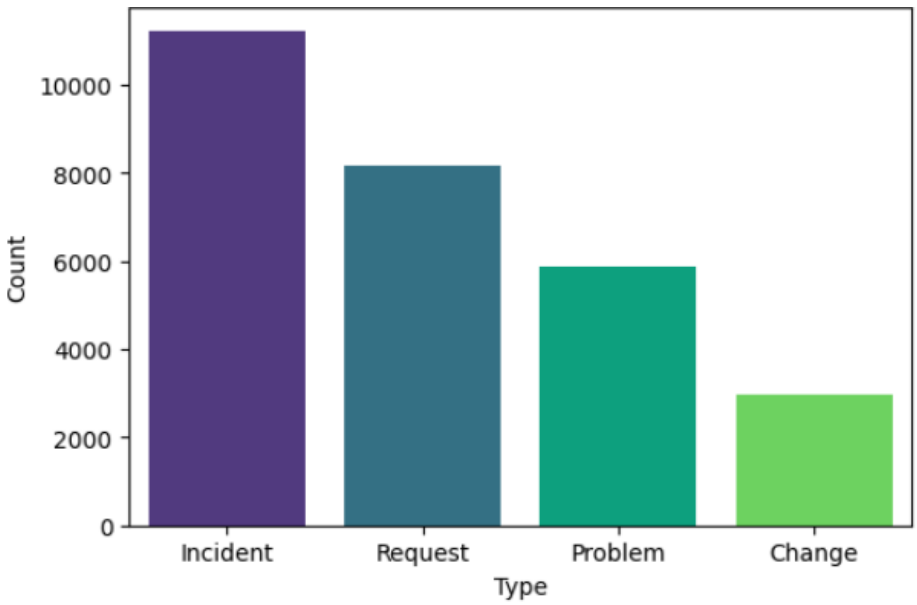

Figure 3.3- Class Distribution Before Oversampling

3.4 Text Cleaning and Preprocessing

The original ticket text contained noise such as URLs, numerical identifiers, email references, punctuation, inconsistent casing, and extra whitespace. To prepare the dataset for modeling, a structured preprocessing pipeline was developed in Python based on **NLTK**, **re**, and **string** libraries.

This cleaning workflow performs the following operations:

Lowercasing

Ensures uniform representation across all tickets.

URL Removal

Removes hyperlinks that do not contribute to classification.

Number Removal

Eliminates numeric identifiers that often represent ticket IDs or timestamps.

Punctuation Removal

Removes special characters and symbols to reduce noise.

Whitespace Normalization

Consolidates multiple spaces and trims leading or trailing spaces.

Stopword Removal

Filters out commonly occurring words like *the*, *is*, *at*, etc., which do not add meaning.

This preprocessing pipeline transforms raw, noisy support messages into clean, concise text suitable for machine learning models.

3.5 BERT Model Architecture

The BERT model is the core component of this system. It uses a transformer-based architecture that can understand the meaning of sentences by analyzing them in both directions (left-to-right and right-to-left). This allows BERT to understand context better than traditional models.

The version used in this project is **bert-base-uncased**, which contains 12 layers of transformer blocks and produces a powerful representation of each input ticket. A classification layer is added on top to predict one of the four classes.

BERT is fine-tuned on the ticket dataset so that it learns how IT support language works. This helps achieve high accuracy and reliable predictions.

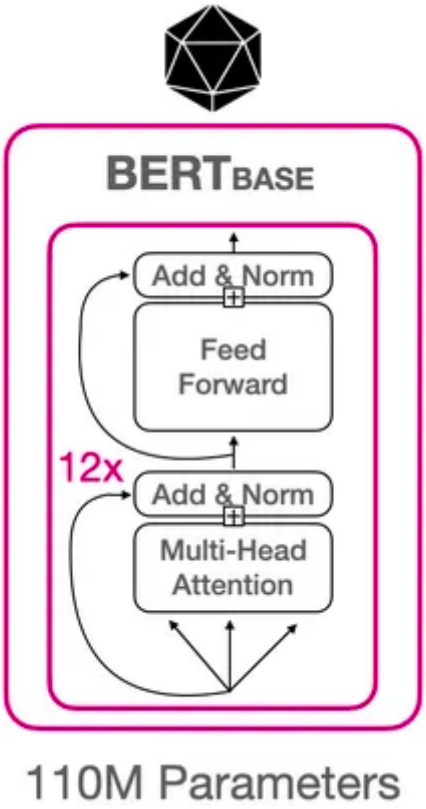

Figure 3.5– BERT Model Architecture

3.6 System Modules and Their Roles

The system is divided into several modules, each responsible for a specific task. A modular structure helps in easy debugging, upgrading, and future improvements.

The major modules include:

- **Data Preprocessing Module** – Cleans and prepares text
- **Model Training Module** – Trains LR, SVM, and BERT models
- **Model Deployment Module** – Hosts BERT using Flask API
- **Automation Module** – Uses n8n to route classified tickets
- **Communication Module** – Sends categorized outputs to Telegram groups

Through coordinated interaction among these modules, the system ensures fully automated ticket processing.

3.7 Summary

This chapter presented the structural design of the automated support ticket classification and routing system. It discussed the dataset, preprocessing pipeline, BERT architecture, and system modules. Together, these components create a solid foundation for implementing a complete end-to-end solution that performs classification and routing without human involvement. The next chapter describes the methodology and technical implementation details behind model training and deployment.

CHAPTER 4

METHODOLOGY

This chapter explains the methodology adopted to build the automated support ticket classification and routing system. It describes the full workflow starting from data preprocessing, model training, evaluation, and deployment. The goal of this methodology is to develop a reliable and efficient end-to-end system that can classify tickets into four categories and automatically route them through workflow automation.

4.1 Overview of the Methodology

The methodology followed in this project involves transforming raw ticket data into a clean, structured form, training multiple machine learning and transformer models, comparing their performance, and integrating the best-performing model into an automated workflow. The overall objective is to create a reliable classification pipeline that can categorize incoming support tickets with high accuracy and minimal delay.

The workflow consists of four major phases:

1. **Data Preparation and Cleaning**
2. **Model Training and Evaluation**
3. **BERT Fine-Tuning for Final Classification**
4. **Deployment and Automation**

Each phase plays an important role in ensuring that the system works efficiently from end to end.

4.2 Baseline Model Training

To establish strong benchmark results, two traditional machine learning models were trained using TF-IDF feature vectors: **Logistic Regression** and **Support Vector Machine**. These models provide a baseline for comparison with the transformer-based model.

4.2.1 Logistic Regression

Logistic Regression was trained using TF-IDF features extracted from the cleaned ticket text. The model handled thousands of tickets and produced consistent performance during evaluation.

Accuracy: 0.7749

Metrics Summary:

- Good precision for frequent classes
- Weak recall for rare classes
- Performance affected by class imbalance

4.2.2 Support Vector Machine (SVM)

SVM was trained using the same TF-IDF representations. Due to its margin-based classification, the SVM model produced results similar to Logistic Regression.

Accuracy: 0.7736

Metrics Summary:

- Strong performance on “Incident” and “Request”
- Lower accuracy on “Problem” and “Change”
- Struggled with multi-sentence ticket descriptions

These baseline models demonstrated the limitations of traditional feature-based classification techniques for complex ticket data.

4.3 BERT Model Fine-Tuning

BERT was selected as the final classification model due to its advanced contextual understanding capabilities. A **bert-base-uncased** model was fine-tuned on the cleaned training dataset with the following key steps:

- Tokenizing ticket text using BERT tokenizer
- Generating attention masks
- Passing tokens into BERT encoder layers
- Adding a dense classification layer
- Fine-tuning all layers during training
- Using AdamW optimizer and learning rate scheduling

The model was trained for multiple epochs until validation metrics stabilized.

BERT Model Accuracy: 0.8961

This score shows a clear improvement over traditional models. BERT successfully captured semantic meaning and contextual patterns found in multi-sentence and technical ticket descriptions.

4.4 Evaluation Metrics

Model evaluation was conducted using four main metrics:

- **Accuracy**
- **Precision**
- **Recall**
- **F1-score**

These metrics provide a balanced understanding of model performance across all ticket categories.

Macro Avg: 0.90

Class	Precision	Recall	F1-score	Support
Change	0.98	0.89	0.98	598
Incident	0.89	0.85	0.87	2243
Problem	0.74	0.80	0.77	1179
Request	0.99	0.99	0.99	1633

4.5 Confusion Matrix

The confusion matrix highlights how well the BERT model distinguished between different classes. It provides insight into which categories were commonly misclassified.

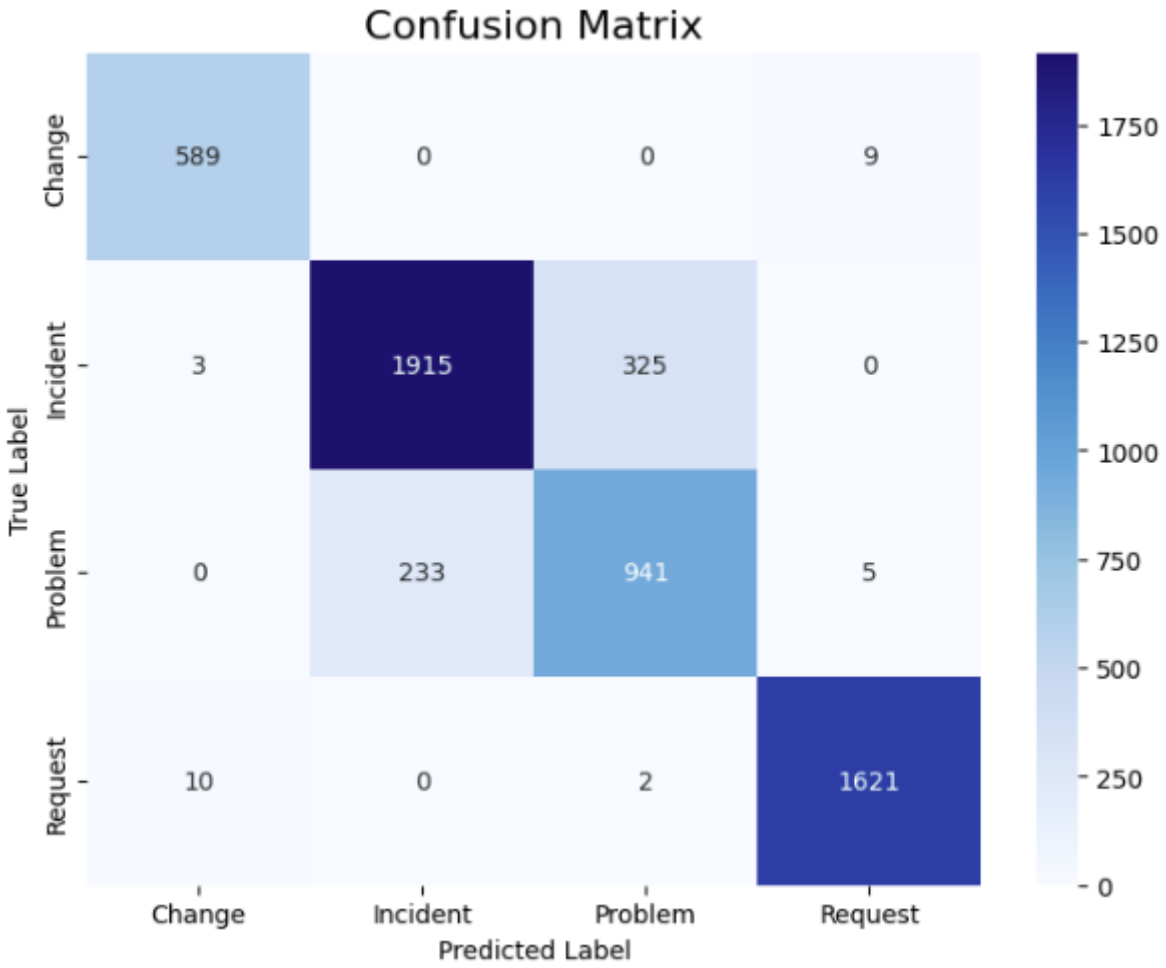

Figure 4.1 – Confusion Matrix of the Fine-Tuned BERT Model

Interpretation :

- Strong diagonal values indicate high correct classification
- Most confusion occurred between *Incident* and *Problem*, which often contain similar wording
- *Request* showed near-perfect classification

4.6 Deployment Using Flask

Once the BERT model achieved optimal performance, it was deployed using a simple Flask API. The deployed API performs the following steps:

1. Accepts ticket text through a POST request
2. Loads the fine-tuned BERT model
3. Processes the text using tokenizer
4. Runs prediction
5. Returns the predicted category

The API acts as a bridge between the model and the automation workflow.

4.7 Workflow Automation Using n8n

The automation pipeline enables full end-to-end processing of tickets. n8n was used due to its flexibility and simple integration with APIs and communication tools.

The workflow includes:

- A webhook trigger to receive incoming tickets
- A call to the Flask API for model prediction
- Formatting the prediction output
- Sending classified tickets to respective Telegram groups
- Logging or storing tickets if necessary

This automation removes the need for manual routing and ensures real-time delivery.

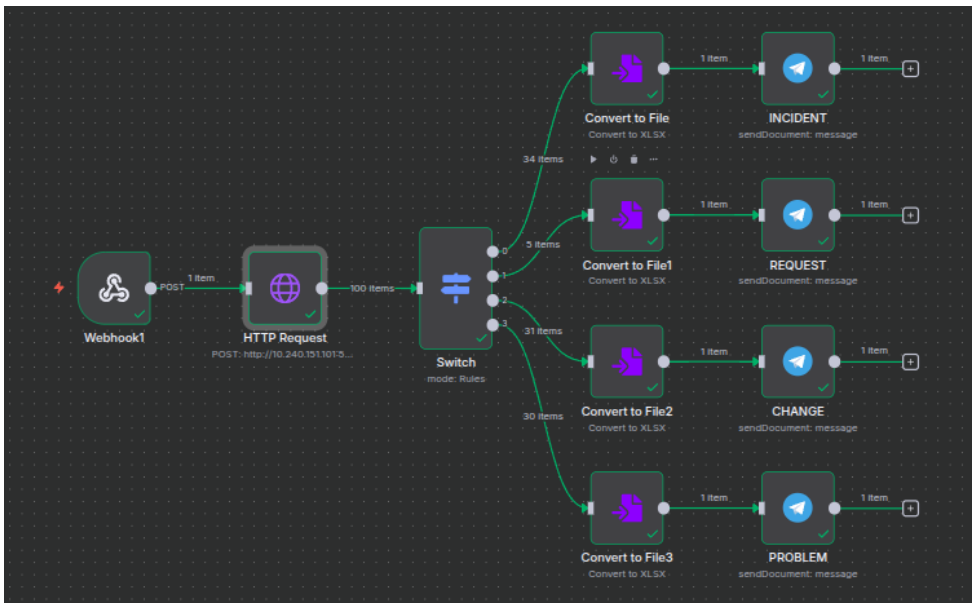

Figure 4.7 – n8n Workflow for Automated Ticket Routing

4.8 Summary

This chapter presented a simplified overview of the methodology followed in the project. It explained the training process for baseline models, the fine-tuning of BERT, evaluation metrics, model performance results, and the deployment and automation strategy. The methodology ensures that the final system achieves high accuracy, reliability, and practical usability for automated ticket classification.

CHAPTER 5

IMPLEMENTATION

This chapter describes the practical implementation of the automated ticket classification and routing system. While earlier chapters discussed the system design and methodology, this chapter focuses on the essential development steps required to make the system function end-to-end. The implementation includes model development, API deployment, workflow automation, and complete system testing.

5.1 Model Implementation

The implementation began with training baseline models—Logistic Regression and Support Vector Machine—using TF-IDF representations extracted from the cleaned dataset. These baseline models were implemented using scikit-learn and served as performance benchmarks.

The core component of the system is the **bert-base-uncased** model, which was fine-tuned using the ticket dataset. The implementation included tokenizing text, generating attention masks, creating DataLoaders, and training the classification layer connected to BERT. The training process continued for several epochs until the model achieved stable performance. The final BERT model achieved an accuracy of **0.8961**, significantly outperforming both baseline models.

5.2 API Development Using Flask

Once the BERT model was trained, it was deployed as a REST API using Flask. The API exposes a single endpoint (`/predict`), which accepts ticket text as input. Upon receiving a request, the API:

1. Tokenizes the input using BERT tokenizer
2. Runs the text through the fine-tuned BERT model
3. Returns the predicted class in JSON format

This API acts as the communication channel between the model and the automation workflow. The lightweight implementation ensures real-time response and simple integration with external systems.

5.3 Workflow Automation Using n8n

After deploying the API, it was integrated into an automated workflow using **n8n**. The automation pipeline ensures every ticket is classified and routed instantly without human involvement. The n8n workflow includes the following main nodes:

- **Webhook Node** – Receives the incoming ticket description
- **HTTP Request Node** – Sends the description to the Flask API
- **Function Node** – Processes and formats the predicted category
- **Telegram Node** – Delivers the ticket to the appropriate support group

This workflow enables a fully automated classification and routing system.

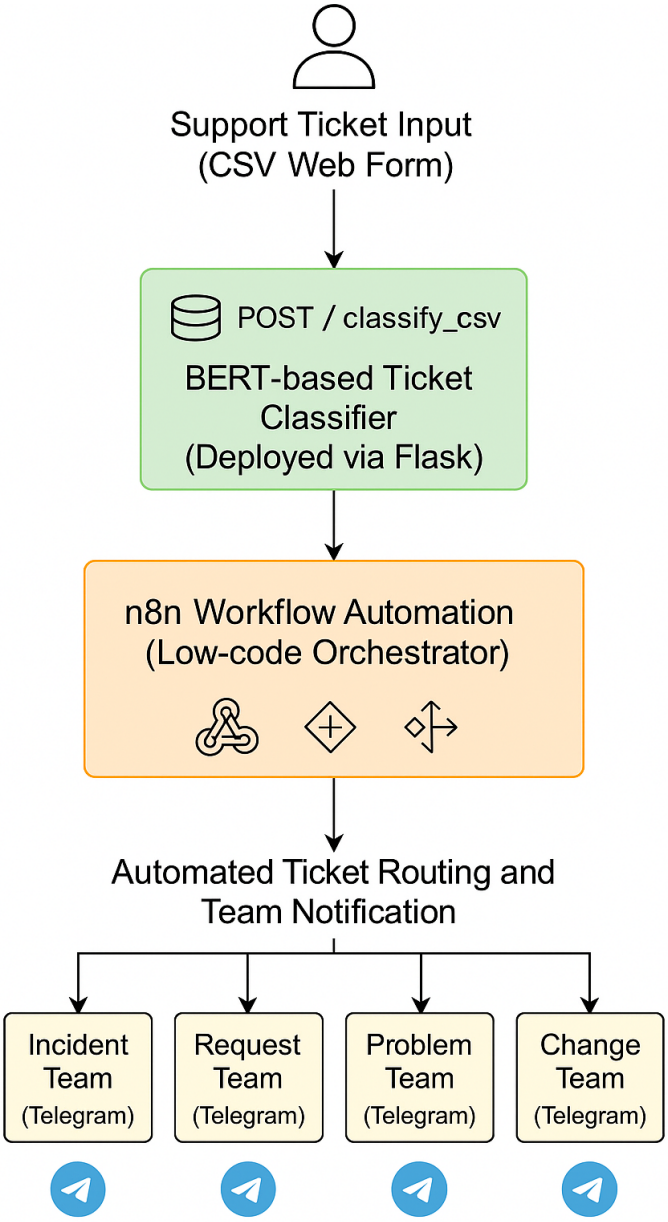

Figure 5.3 – n8n Workflow

5.4 System Integration and Testing

All components—including data cleaning scripts, the BERT model, the Flask API, and the n8n workflow—were integrated into a single operational pipeline. Testing was performed by sending sample ticket descriptions through the webhook and verifying each stage:

1. The ticket reached the API successfully
2. The BERT model produced the correct prediction
3. The n8n workflow routed the result to the correct Telegram group
4. Final message formatting was clear and consistent

Multiple test cases were used to ensure the system handled variations such as long descriptions, short messages, and technical terminology. The integrated system performed accurately and efficiently across all tests.

5.5 User Interface and Output Screens

The developed system provides two ways to interact with the ticket classification model. The first interface allows direct input of ticket text for instant prediction, while the second interface enables batch processing using Excel files through the n8n workflow. These interfaces make the system flexible for both individual and bulk ticket processing.

5.5.1 Direct Ticket Classification Interface

This interface allows users to directly enter ticket text and obtain the predicted category in real time. The input is sent to the Flask API, where the BERT model processes the text and returns the classification result instantly.

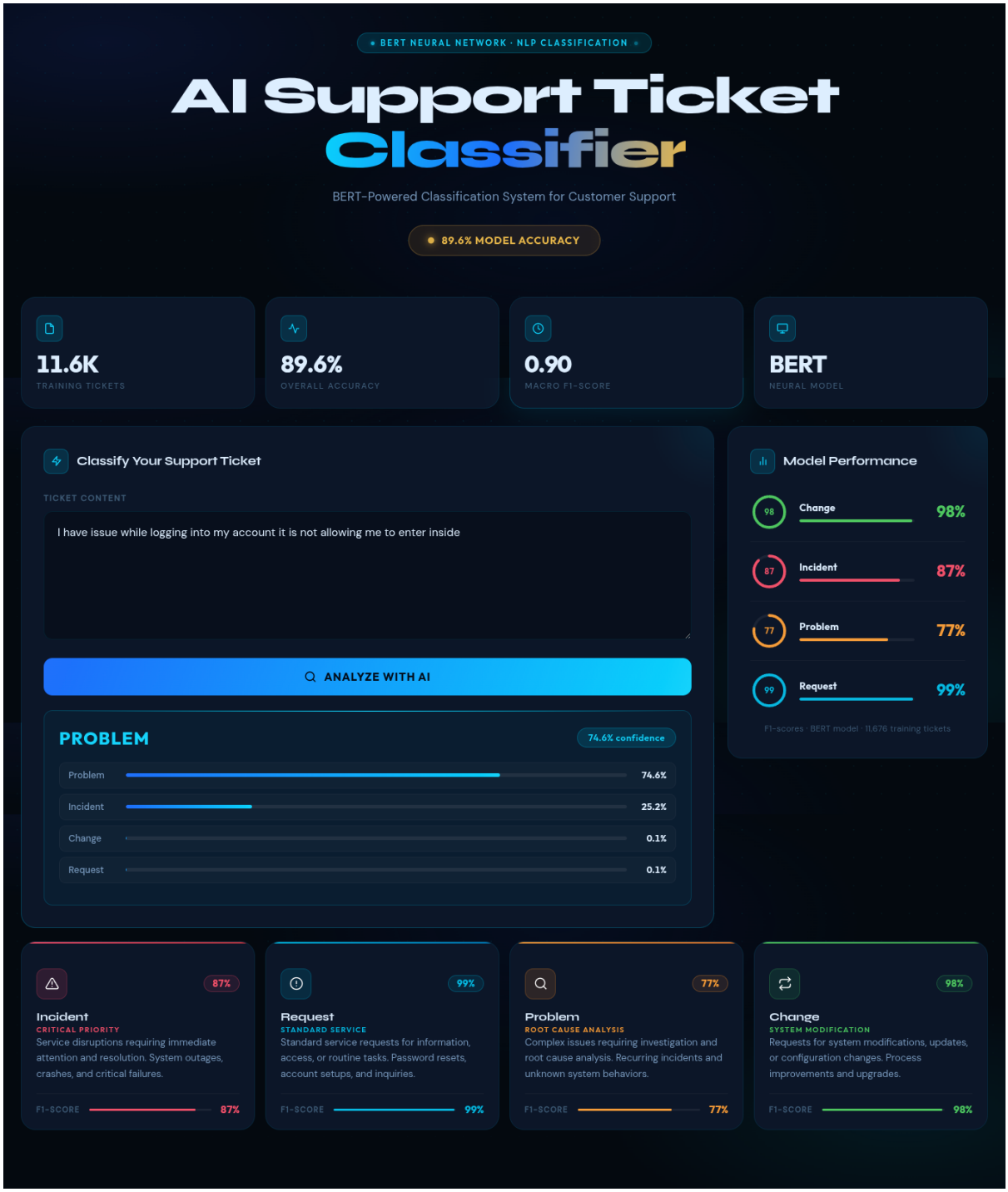

Figure 5.5.1 - Direct Ticket Classification Interface

5.5.2 n8n Form Submission for Batch Ticket Processing

This interface allows users to directly enter ticket text or you can upload the xecel to get the multiple tickets processed simultaneously.the image despicts the form submission in the n8n workflow at the start.

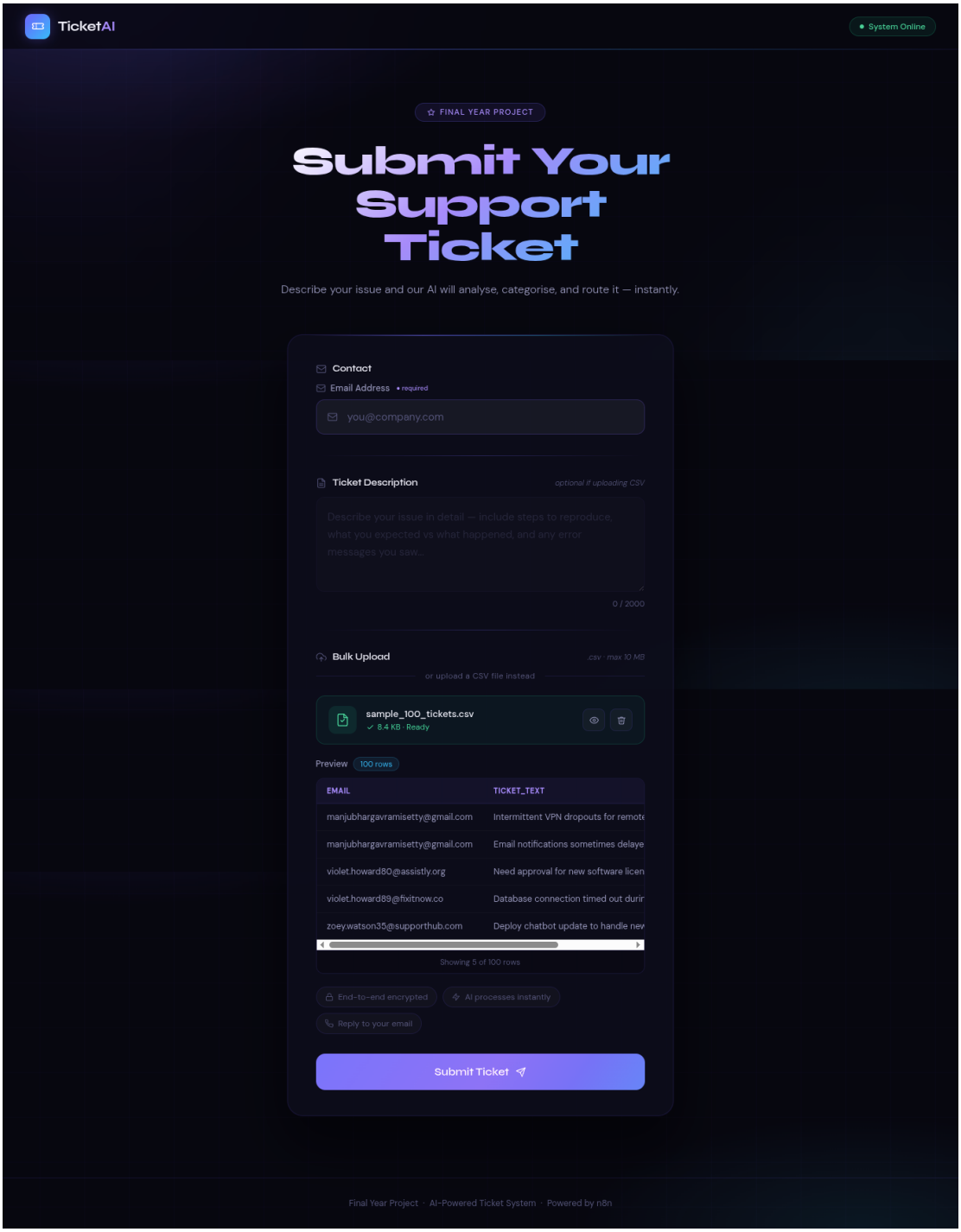

Figure 5.5.2 n8n form submission for Batch Ticket Processing

5.6 Summary

This chapter outlined the essential steps involved in implementing the automated support ticket classification system. It detailed the training of models, deployment of the BERT classifier using Flask, workflow automation using n8n, and complete system testing. The implementation ensures a fully automated, reliable, and scalable solution for real-time ticket processing.

CHAPTER 6

RESULTS AND EVALUATION

This chapter presents the experimental results obtained from the baseline machine learning models and the fine-tuned BERT model. The purpose of this evaluation is to compare model performances and demonstrate the effectiveness of BERT in accurately classifying support tickets into four categories: Incident, Problem, Change, and Request. The evaluation metrics used include accuracy, precision, recall, and F1-score, which provide a comprehensive understanding of model strengths and weaknesses.

6.1 Evaluation Metrics

To assess the performance of all models consistently, four standard classification metrics were used:

1. **Accuracy** – Measures the overall correctness of predictions

2. **Precision** – Measures how many predicted positives were actually correct

$$\text{Precision} = TP/(TP+ FP)$$

3. **Recall** – Measures how many actual positives were identified correctly

$$\text{Recall} =TP/(TP+ F N)$$

4. **F1-score** – Harmonic mean of precision and recall

$$\text{F1 Score} =2 (\text{Precision} \mathbf{X} \text{Recall})/(\text{Precision}+ \text{Recall})$$

These metrics are widely used in NLP classification tasks and are suitable for multi-class evaluation.

6.2 Baseline Model Results

The first part of the evaluation involved training and testing two traditional machine learning models: Logistic Regression (LR) and Support Vector Machine (SVM). Both models used TF-IDF vectors generated from the cleaned ticket descriptions.

6.2.1 Logistic Regression Performance

Class	Precision	Recall	F1-score	Support
Change	0.90	0.92	0.91	496
Incident	0.79	0.66	0.72	1925
Problem	0.50	0.67	0.57	1001
Request	0.97	0.97	0.97	1367
Accuracy			**0.7749**	4789

Logistic Regression performed well on the *Request* class but struggled with the *Problem* and *Incident* classes.

6.2.2 Support Vector Machine Performance

Class	Precision	Recall	F1-score	Support
Change	0.92	0.92	0.92	496
Incident	0.80	0.64	0.71	1925
Problem	0.50	0.69	0.58	1001
Request	0.97	0.97	0.97	1367
Accuracy			**0.7736**	4789

SVM achieved accuracy similar to Logistic Regression and showed the same pattern of weak recall for minority classes.

6.3 BERT Model Performance

The fine-tuned BERT model significantly outperformed both baseline models. Its ability to understand context and semantic meaning resulted in higher accuracy and more balanced F1-scores across all classes.

Class	Precision	Recall	F1-score	Support
Change	0.98	0.89	0.98	598
Incident	0.89	0.85	0.87	2243
Problem	0.74	0.80	0.77	1179
Request	0.99	0.99	0.99	1633
Accuracy			**0.8961**	5653

Compared to the baseline models, BERT improves accuracy by more than 11%, demonstrating the effectiveness of transformer-based learning for ticket classification.

6.4 Confusion Matrix Analysis

The confusion matrix of the BERT model helps visualize model performance by showing the correct and incorrect classifications for each class.

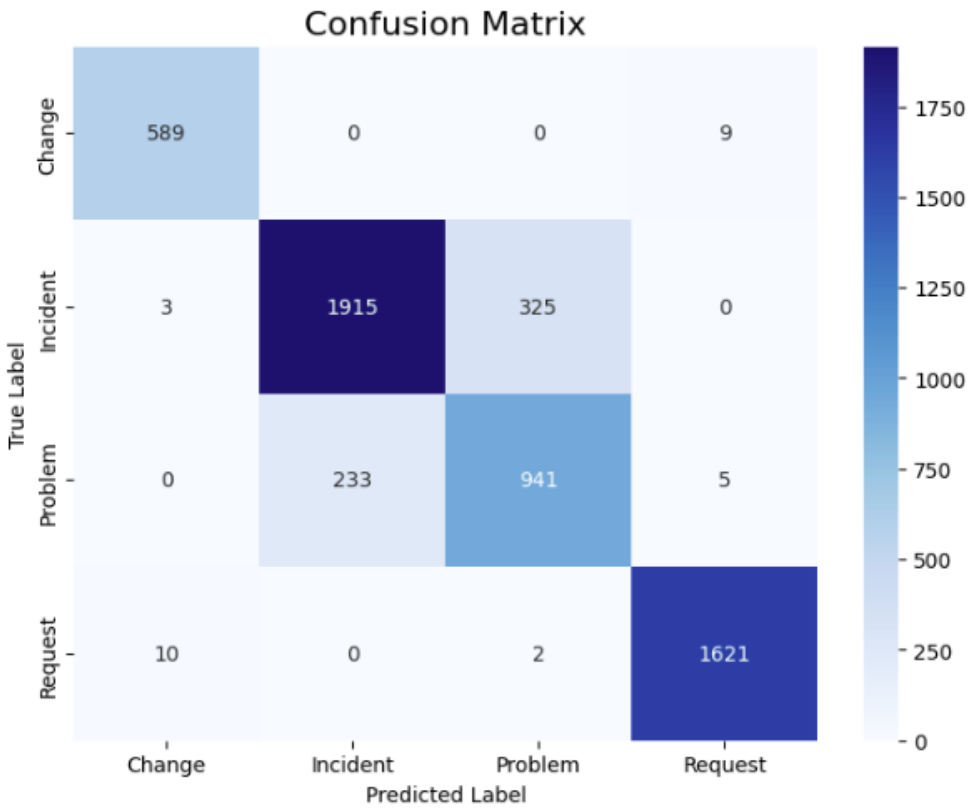

Figure 6.4 – Confusion Matrix of BERT Model

Interpretation :

- Strong diagonal values indicate high accuracy across all classes.
- The *Request* class shows near-perfect predictions.
- Some misclassification occurs between *Incident* and *Problem* due to overlap in wording.
- Overall, the model maintains strong performance across all categories.

6.5 Comparative Analysis

To clearly demonstrate improvement, the accuracy of all three models is summarized below.

Model	Accuracy
Logistic Regression	0.7749
SVM	0.7736
BERT	**0.8961**

From the results, it is evident that BERT outperforms the baseline models by a significant margin due to its contextual understanding and ability to process long, descriptive ticket text.

6.6 Summary

This chapter presented the results obtained from various models trained for ticket classification. While traditional machine learning models delivered moderate accuracy, the transformer-based BERT model achieved superior performance across all evaluation metrics. The confusion matrix further confirmed BERT's robustness in real-world ticket classification scenarios. The findings validate the choice of BERT as the final model used in the automated ticket classification and routing system.

CHAPTER 7
CONCLUSION AND FUTURE WORK

7.1 Conclusion

This project successfully developed an automated support ticket classification and routing system using Natural Language Processing (NLP) and workflow automation. The main objective was to classify support tickets accurately into four predefined categories—Incident, Problem, Change, and Request—and automatically route them to the appropriate support team.

The system was implemented through a combination of baseline machine learning models and a transformer-based BERT model. While Logistic Regression and SVM provided moderate performance, the fine-tuned BERT model achieved significantly better accuracy, reaching **0.8961**. This performance improvement demonstrates the effectiveness of transformer models in understanding complex and context-rich ticket descriptions.

The trained BERT model was deployed using a Flask API, which serves as the prediction engine for real-time classification. This API was integrated with an **n8n workflow**, enabling fully automated routing of classified tickets to designated Telegram groups. The complete pipeline—from receiving a ticket, classifying it, and forwarding the result—is fully automated with no manual intervention required.

Overall, the system achieves the intended goal of providing a fast, accurate, and scalable solution for IT support environments, reducing manual workload and improving operational efficiency.

7.2 Future Work

Although the system performs effectively, there are several enhancements that can be incorporated in future work:

1. Multi-label Ticket Classification

Some real-world support tickets may belong to multiple categories at once. Extending the model to support multi-label classification can improve flexibility.

2. Priority Prediction

In addition to classification, predicting the priority level (Low, Medium, High) can help automate escalation management.

3. Integration with Additional Communication Platforms

Currently, the system routes tickets to Telegram. Future work may include integration with platforms like Slack, Microsoft Teams, or email automation.

4. Deployment on Cloud Services

Hosting the Flask API and workflow on cloud platforms (AWS, Azure, GCP) can improve scalability and performance under heavy workloads.

5. Model Optimization and Distillation

BERT is powerful but computationally heavy. Techniques such as model distillation or switching to lighter transformer variants (DistilBERT, ALBERT) can reduce latency.

6. Advanced Text Preprocessing

Incorporating domain-specific entity extraction, spell correction, or contextual paraphrasing may further improve classification accuracy.

7. Dashboard for Monitoring

A web-based dashboard showing daily ticket counts, class distribution, model performance, and system logs would enhance usability for IT teams.

REFERENCES

1. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). **BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.** *NAACL-HLT 2019.* Available: https://arxiv.org/abs/1810.04805
2. Truss, M., & Böhm, S. (2024). **AI-based Classification of Customer Support Tickets: State of the Art and Implementation with AutoML.** arXiv:2406.01789. Available: https://arxiv.org/abs/2406.01789
3. **n8n Automation Tool.** n8n GmbH. Available: https://n8n.io/
4. **Flask Documentation.** Pallets Projects. Available: https://flask.palletsprojects.com/
5. **Transformers Library.** Hugging Face. Available: https://huggingface.co/transformers/
6. **Scikit-learn: Machine Learning in Python.** scikit-learn developers. Available: https://scikit-learn.org/
7. **PyTorch Documentation.** PyTorch Foundation. Available: https://pytorch.org/
8. **Pandas Documentation** The Pandas Development Team. Available: https://pandas.pydata.org/